

1 Bitwise Trick

1.1 tricks

```

1 bool isOdd(int x) {
2     return x & 1;
3 }
4
5 int isolateRightmostSetBit(int x) {
6     return x & -x; // 取得最右邊的 1
7 }
8
9 bool isPowerOfTwo(int x) {
10    return x > 0 && (x & (x - 1)) == 0;
11 }
12
13 int countSetBits(int x) {
14     int count = 0;
15     while (x) {
16         x &= (x - 1); // 清除最右邊的 1
17         count++;
18     }
19     return count;
20 }
21
22 int turnOffRightmostSetBit(int x) {
23     return x & (x - 1); // 將最右邊的 1 設為 0
24 }
25
26 int setBit(int x, int pos) {
27     return x | (1 << pos); // 設該位元為 1
28 }
29
30 int clearBit(int x, int pos) {
31     return x & ~(1 << pos); // 設該位元為 0
32 }
33
34 int toggleBit(int x, int pos) {
35     return x ^ (1 << pos); // 切換該位元
36 }
37
38 int getBit(int x, int pos) {
39     return (x >> pos) & 1; // 取得該位元值
40 }
41
42 int modPowerOfTwo(int x, int mod) {
43     return x & (mod - 1); // x % mod, mod 必須
44     // 是 2 的幕次
45 }
46
47 // a + b = (a ^ b) + 2 * (a & b)
48 int add(int a, int b) {
49     while (b != 0) {
50         int carry = a & b; // 計算進位
51         a = a ^ b; // 無進位相加
52         b = carry << 1; // 將進位左移一位
53     }
54     return a;
55 }
56
57 // a - b = (a ^ b) - 2 * ((~a) & b)
58 int subtract(int a, int b) {

```

```

57     while (b != 0) {
58         int borrow = (~a) & b; // 計算借位
59         a = a ^ b; // 無借位相減
60         b = borrow << 1; // 將借位左移一位
61     }
62     return a;
63 }
64 // a * b = sum(a << i) for each set bit i in b
65 int multiply(int a, int b) {
66     int result = 0;
67     while (b > 0) {
68         if (b & 1) { // 如果 b 的最低位是 1
69             result = add(result, a); // 使用
70             // 上面的加法函數
71         }
72         a <<= 1; // 將 a 左移一位 (相當於乘以 2)
73         b >>= 1; // 將 b 右移一位 (相當於除以 2)
74     }
75     return result;
76 }
77 // a / b = sum(1 << i) for each set bit i in a
78 int divide(int a, int b) {
79     if (b == 0) throw std::invalid_argument("Division by zero");
80     int result = 0;
81     int power = 1;
82     int tempB = b;
83     while (tempB <= a && tempB > 0) { // 防止溢位
84         tempB <<= 1;
85         power <<= 1;
86     }
87     while (power > 1) {
88         tempB >>= 1;
89         power >>= 1;
90         if (a >= tempB) {
91             a = subtract(a, tempB); // 使用
92             // 上面的減法函數
93             result = add(result, power); // 使用上面的加法函數
94         }
95     }
96     return result;
97 }
98
99 int findUnique(const std::vector<int>& nums) {
100    {
101        int unique = 0;
102        for (int num : nums) {
103            unique ^= num; // XOR 相同的數字會抵消
104        }
105        return unique;
106    }
107 }
108
109 int graycode(int n) { // n 轉 Gray code

```

```

108     return n ^ (n >> 1);
109 }

```

2 DP

2.1 Digit DP

```

1 string num;
2
3 ll dp[the max len of num + 1][2][2][...];
4 memset(dp, -1, sizeof(dp));
5
6 ll count(ll pos, bool tight, bool leadingZero, ...) {
7     if (dp[pos][tight][leadingZero][...] != -1) {
8         return dp[pos][tight][leadingZero][...];
9     }
10
11     if (base-case) { // e.g. pos == num.size() or other base case condition
12         // do something
13     }
14
15     ll res = 0;
16     ll up = (tight ? num[pos] - '0' : 9);
17     for (ll d = 0; d <= up; ++d) {
18         res += count(
19             pos + 1,
20             tight and (d == num[pos] - '0'),
21             leadingZero and (d == 0),
22             ...
23         );
24     }
25
26     return dp[pos][tight][leadingZero][...] = res;
27 }
28
29 count(0, true, true, ...) // the answer
30
31 // =====
32 // =====
33
34 // AtCoder ABC154E
35 // Find the number of integers between 1 and N (inclusive)
36 // that contains exactly K non-zero digits when written in base ten.
37 // 1 <= N <= 10^100, 1 <= K <= 3
38
39 #include <bits/stdc++.h>
40 using namespace std;
41 using ll = long long;
42
43 ll k;
44 string num;
45
46 ll dp[102][2][4];

```

```

48 ll count(ll pos, bool tight, ll cnt) {
49     if (dp[pos][tight][cnt] != -1) {
50         return dp[pos][tight][cnt];
51     }
52
53     if (cnt == 0) return dp[pos][tight][cnt] = 1;
54     else if (pos == num.size()) return dp[pos][tight][cnt] = 0;
55
56     ll res = 0;
57     ll up = (tight ? num[pos] - '0' : 9);
58     for (ll d = 0; d <= up; ++d) {
59         res += count(
60             pos + 1,
61             tight and (d == num[pos] - '0'),
62             cnt + (d == 0 ? 0 : -1)
63         );
64     }
65
66     return dp[pos][tight][cnt] = res;
67 }
68
69 int main() {
70     ios::sync_with_stdio(false);
71     cin.tie(nullptr);
72
73     cin >> num >> k;
74
75     memset(dp, -1, sizeof(dp));
76
77     cout << count(0, true, k) << '\n';
78
79     return 0;
80 }

```

2.2 Interval DP

```

1 ll n; // the length of the array
2
3 vector<ll> a(n + 1);
4
5 vector<vector<ll>> dp(n + 1, vector<ll>(n + 1));
6
7 for (ll i = 1; i <= n; ++i) {
8     dp[i][i] = "initial value"; // length 1 interval [i, i]
9 }
10 for (ll len = 2; len <= n; ++len) { // loop through all length from 2 to n
11     for (ll l = 1; l <= n - len + 1; ++l) {
12         ll r = l + len - 1;
13
14         dp[l][r] = ...; // interval [l, r]
15     }
16 }
17
18 dp[1][n] // the answer (interval [1, n])

```

2.3 Knapsack

```

1 // 0/1 knapsack
2
3 const ll INF = 9e18;
4
5 ll n, wmx; cin >> n >> wmx;
6 vector<ll> w(n + 1); for (ll i = 1; i <= n;
7 ++i) cin >> w[i];
8 vector<ll> v(n + 1); for (ll i = 1; i <= n;
9 ++i) cin >> v[i];
10
11 vector<ll> dp(wmx + 1, -INF); dp[0] = 0;
12 for (ll i = 1; i <= n; ++i) {
13     for (ll j = wmx; j >= w[i]; --j) {
14         dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
15     }
16 }
17
18 ll mx = dp[0];
19 for (ll j = 1; j <= wmx; ++j) {
20     mx = max(mx, dp[j]);
21 }
22 cout << mx << '\n';
23
24 // -----
25 // Unbounded
26 // just change the inner for loop to the
    following line
27 // for (ll j = w[i]; j <= wmx; ++j) ...

```

2.4 Subset DP

```

1 for (ll s = 1; s < (1LL << n); ++s) {
2     for (ll sbs = s; sbs > 0; sbs = ((sbs - 1)
3         & s)) {
4         ll c = s - sbs; // ll c = sbs^s; //
            this one is equivalent
5     }
6 }
7 // O(3^n), n <= 16
8 // sbs 是降序的 s 的子集
9 // c 是升序的 s 的子集

```

2.5 LIS

```

1 ll LISLength(const vector<ll>& arr) {
2     vector<ll> d;
3     for (ll x : arr) {
4         auto it = lower_bound(d.begin(), d.
5             end(), x);
6         if (it == d.end()) d.push_back(x);
7         else *it = x;
8     }
9     return d.size();
10 }

```

```

11 // find LIS sequence
12 for (ll i = 0; i < num.size(); ++i) {
13     if (lis.empty() || lis.back() < num[i]) {
14         lis.push_back(num[i]);
15         dp[i] = lis.size();
16     }
17     else {
18         auto iter = lower_bound(all(lis), num
19             [i]);
20         dp[i] = iter - lis.begin() + 1;
21         *iter = num[i];
22     }
23 }
24 ll length = lis.size();
25 for (ll i = num.size() - 1; i >= 0; --i) {
26     if (dp[i] == length) {
27         ans.push_back(num[i]);
28         length -= 1;
29     }
30 }

```

2.6 LCS

```

1 ll n; cin >> n;
2 string s; cin >> s; s = " " + s;
3 ll m; cin >> m;
4 string t; cin >> t; t = " " + t;
5
6 vector<vector<ll>> dp(n + 1, vector<ll>(m +
7     1, 0));
8 for (ll i = 1; i <= n; ++i) {
9     for (ll j = 1; j <= m; ++j) {
10         if (s[i] == t[j]) {
11             dp[i][j] = dp[i - 1][j - 1] + 1;
12         }
13         else {
14             dp[i][j] = max(dp[i - 1][j], dp[i][j -
15                 1]);
16         }
17     }
18 }
19 cout << dp[n][m] << '\n'; // the length of
    LCS

```

3 D&C

3.1 MergeSort Finds the Number of Inversions

```

1 void merge(vector<int>& arr, ll l, ll r, ll
2     x, ll y) {
3     ll left = l;
4     vector<ll> tmp; tmp.reserve(y - l + 1);
5     while (l <= r and x <= y) {
6         if (arr[l] <= arr[x]) {
7             tmp.push_back(arr[l++]);
8         }
9     }
10 }

```

```

7     }
8     else {
9         tmp.push_back(arr[x++]);
10    }
11 }
12 while (l <= r) {
13     tmp.push_back(arr[l++]);
14 }
15 while (x <= y) {
16     tmp.push_back(arr[x++]);
17 }
18 for (ll i = left; i <= y; ++i) {
19     arr[i] = tmp[i - left];
20 }
21 }
22 mergeSort(vector<ll>& arr, ll l, ll r) {
23     if (l == r) return 0;
24     ll mid = l + (r - l) / 2;
25     ll lcmt = mergeSort(arr, l, mid);
26     ll rcmt = mergeSort(arr, mid + 1, r);
27
28     // ----- main logic -----
29     ll cnt = lcmt + rcmt;
30     ll a = l, b = mid, c = mid + 1, d = r;
31     // c is the current checking
        position
32     while (a <= b) {
33         while (c <= d and arr[a] > arr[c]) c
34             += 1;
35         cnt += c - (mid + 1);
36         a += 1;
37     }
38     // -----
39     merge(arr, l, mid, mid + 1, r);
40
41     return cnt;
42 }

```

4 Data Structure

4.1 DSU

```

1 ll cc;
2 vector<ll> djs, sz;
3 ll find(ll u) {
4     if (u == djs[u]) return u;
5     return djs[u] = find(djs[u]);
6 }
7 void join(ll u, ll v) {
8     u = find(u);
9     v = find(v);
10    if (u == v) return; // don't forgot
        this line
11    if (sz[u] < sz[v]) swap(u, v);
12    djs[v] = u;
13    sz[u] += sz[v];
14    cc -= 1;
15 }
16 void init(ll n) {
17     djs.clear(); djs.resize(n + 1);
18 }

```

```

18     for (ll i = 1; i <= n; ++i) djs[i] = i;
19     sz.clear(); sz.resize(n + 1, 1);
20     cc = n;
21 }

```

4.2 Segment Tree

```

1 // CSES Range Updates and Sums
2 // 1. Increase each value in range [a,b] by
    x.
3 // 2. Set each value in range [a,b] to x.
4 // 3. Calculate the sum of values in range [
    a,b].
5
6 #include <bits/stdc++.h>
7 using namespace std;
8 using ll = long long;
9
10 #define lson 2*n + 1
11 #define rson 2*n + 2
12
13 class Node {
14 public:
15     ll val;
16     ll setVal;
17     ll addVal;
18     bool isSet;
19 };
20
21 ll sz, q;
22 vector<ll> a;
23 vector<Node> nds;
24
25 void build(ll l, ll r, ll n = 0) {
26     if (l == r) {
27         nds[n].val = a[l];
28         nds[n].setVal = 0;
29         nds[n].addVal = 0;
30         nds[n].isSet = false;
31         return;
32     }
33     ll mid = l + (r - l) / 2;
34     build(l, mid, lson);
35     build(mid + 1, r, rson);
36     nds[n].val = nds[lson].val + nds[rson].
        val;
37 }
38
39 void push(ll l, ll r, ll n) {
40     ll mid = l + (r - l) / 2;
41
42     if (nds[n].isSet) {
43         nds[lson].val = nds[n].setVal * (mid -
44             l + 1);
45         nds[lson].setVal = nds[n].setVal;
46         nds[lson].addVal = 0;
47         nds[lson].isSet = true;
48
49         nds[rson].val = nds[n].setVal * (r - (
50             mid + 1) + 1);
51         nds[rson].setVal = nds[n].setVal;
52         nds[rson].addVal = 0;
53     }
54 }

```

```

52     nds[rson].isSet = true;
53
54     nds[n].isSet = false;
55 }
56
57 nds[lson].val += nds[n].addVal*(mid - 1
58 + 1);
59 nds[lson].addVal += nds[n].addVal;
60
61 nds[rson].val += nds[n].addVal*(r - (mid
62 + 1) + 1);
63 nds[rson].addVal += nds[n].addVal;
64
65 nds[n].addVal = 0;
66 }
67
68 void setVal(ll x, ll y, ll val, ll l, ll r,
69 ll n = 0) {
70     if (l == x and r == y) {
71         nds[n].val = val*(y - x + 1);
72         nds[n].setVal = val;
73         nds[n].addVal = 0;
74         nds[n].isSet = true;
75         return;
76     }
77     push(1, r, n);
78     ll mid = 1 + (r - 1)/2;
79     if (y <= mid) {
80         setVal(x, y, val, l, mid, lson);
81     } else if (x >= mid + 1) {
82         setVal(x, y, val, mid + 1, r, rson);
83     } else {
84         setVal(x, mid, val, l, mid, lson);
85         setVal(mid + 1, y, val, mid + 1, r,
86             rson);
87     }
88     nds[n].val = nds[lson].val + nds[rson].
89     val;
90 }
91
92 void addVal(ll x, ll y, ll val, ll l, ll r,
93 ll n = 0) {
94     if (l == x and r == y) {
95         nds[n].val += val*(y - x + 1);
96         nds[n].addVal += val;
97         return;
98     }
99     push(1, r, n);
100     ll mid = 1 + (r - 1)/2;
101     if (y <= mid) {
102         addVal(x, y, val, l, mid, lson);
103     } else if (x >= mid + 1) {
104         addVal(x, y, val, mid + 1, r, rson);
105     } else {
106         addVal(x, mid, val, l, mid, lson);
107         addVal(mid + 1, y, val, mid + 1, r,
108             rson);
109     }
110     nds[n].val = nds[lson].val + nds[rson].
111     val;
112 }
113
114 ll query(ll x, ll y, ll l, ll r, ll n = 0) {
115     if (l == x and r == y) return nds[n].val
116     ;
117     push(1, r, n);
118     ll mid = 1 + (r - 1)/2;
119     if (y <= mid) {
120         return query(x, y, l, mid, lson);
121     } else if (x >= mid + 1) {
122         return query(x, y, mid + 1, r, rson)
123     ;
124     } else {
125         ll a = query(x, mid, l, mid, lson);
126         ll b = query(mid + 1, y, mid + 1, r,
127             rson);
128         return a + b;
129     }
130 }
131
132 int main() {
133     ios::sync_with_stdio(false);
134     cin.tie(nullptr);
135
136     cin >> sz >> q;
137
138     a.resize(sz + 1);
139     for (ll i = 1; i <= sz; ++i) cin >> a[i]
140     ;
141
142     nds.resize(4*sz);
143     build(1, sz);
144
145     while (q--) {
146         ll act; cin >> act;
147         if (act == 1) {
148             ll l, r, val; cin >> l >> r >>
149             val;
150             addVal(l, r, val, 1, sz);
151         } else if (act == 2) {
152             ll l, r, val; cin >> l >> r >>
153             val;
154             setVal(l, r, val, 1, sz);
155         } else if (act == 3) {
156             ll l, r; cin >> l >> r;
157             cout << query(l, r, 1, sz) << '\n'
158             ;
159         }
160     }
161     return 0;
162 }

```

4.3 BIT

```

1 inline ll lowbit(ll x) {
2     return x & -x;
3 }
4
5 ll n;
6 vector<ll> a, bit;
7
8 void add(ll pos, ll val) {
9     for (ll i = pos; i <= n; i += lowbit(i))
10         bit[i] += val;
11 }
12
13

```

```

14 void init() {
15     for (ll i = 1; i <= n; ++i) {
16         add(i, a[i]);
17     }
18 }
19
20 ll query(ll pos) { // [1, pos]
21     ll sum = 0;
22     for (ll i = pos; i >= 1; i -= lowbit(i))
23         sum += bit[i];
24     return sum;
25 }
26

```

4.4 Treap

```

1 // CSES Cut and Paste
2 // Given a string, your task is to process
3 // operations
4 // where you cut a substring and paste it to
5 // the end of the string.
6 // What is the final string after all the
7 // operations?
8
9 #include <bits/stdc++.h>
10 using namespace std;
11 using ll = long long;
12 using pll = pair<ll, ll>;
13 template<class T> using vec = vector<T>;
14 #define ff first
15 #define ss second
16 #define pb push_back
17 #define rep(n) for (ll _ = 1; _ <= n; ++_)
18
19 // -----
20 mt19937 rng(chrono::steady_clock::now().
21     time_since_epoch().count());
22
23 class Node {
24 public:
25     Node* l, * r;
26     ll val, pri, sz;
27     Node(ll _val) {
28         l = r = nullptr;
29         val = _val;
30         pri = rng();
31         sz = 1;
32     }
33     void pull() {
34         sz = 1;
35         if (l) sz += l->sz;
36         if (r) sz += r->sz;
37     }
38     Node* merge(Node* a, Node* b) {
39         if (!a || !b) return a ? a : b;
40         if (a->pri < b->pri) {
41             a->r = merge(a->r, b);

```

```

41         a->pull();
42         return a;
43     } else {
44         b->l = merge(a, b->l);
45         b->pull();
46         return b;
47     }
48 }
49
50 void splsz(Node* T, Node*& a, Node*& b, ll s
51 ) {
52     if (!T) {
53         a = b = nullptr;
54         return;
55     }
56     ll lsz = T->l ? T->l->sz : 0;
57     if (lsz + 1 <= s) {
58         a = T;
59         splsz(T->r, a->r, b, s - (lsz + 1));
60         a->pull();
61     } else {
62         b = T;
63         splsz(T->l, a, b->l, s);
64         b->pull();
65     }
66 }
67
68 void move_to_tail(Node*& root, ll l, ll r) {
69     // 1-indexed
70     Node* left, * mid, * right;
71     splsz(root, left, mid, l - 1);
72     splsz(mid, mid, right, r - l + 1);
73     root = merge(left, right);
74     root = merge(root, mid);
75 }
76
77 void inorder(Node* root, string& res) {
78     if (!root) return;
79     inorder(root->l, res);
80     res += char(root->val);
81     inorder(root->r, res);
82 }
83
84 int main() {
85     ios::sync_with_stdio(false);
86     cin.tie(nullptr);
87
88     ll n, q;
89     cin >> n >> q;
90
91     string s;
92     cin >> s;
93
94     Node* root = nullptr;
95     for (ll i = 0; i < n; ++i) {
96         root = merge(root, new Node(s[i]));
97     }
98
99     rep (q) {
100         ll l, r;
101         cin >> l >> r;
102         move_to_tail(root, l, r);
103     }
104
105     string res = "";

```

```

105   inorder(root, res);
106   cout << res << '\n';
107
108   return 0;
109 }
110
111 //
112 // =====
113 // CSES Substring Reversals
114 // Given a string, your task is to process
115 // operations
116 // where you reverse a substring of the
117 // string.
118 // What is the final string after all the
119 // operations?
120
121 #include <bits/stdc++.h>
122 using namespace std;
123 using ll = long long;
124 using pll = pair<ll, ll>;
125 template<class T> using vec = vector<T>;
126 #define ff first
127 #define ss second
128 #define pb push_back
129 #define rep(n) for (ll _ = 1; _ <= n; ++_)
130
131 // -----
132
133 mt19937 rng(chrono::steady_clock::now().
134   time_since_epoch().count());
135
136 class Node {
137 public:
138   Node* l,* r;
139   ll val, pri, sz;
140   bool rev_tag;
141   Node(ll _val) {
142     l = r = nullptr;
143     val = _val;
144     pri = rng();
145     sz = 1;
146     rev_tag = false;
147   }
148   void pull() {
149     sz = 1;
150     if (l) sz += l->sz;
151     if (r) sz += r->sz;
152   }
153   void push() {
154     if (rev_tag) {
155       swap(l, r);
156       if (l) l->rev_tag = !l->rev_tag;
157       if (r) r->rev_tag = !r->rev_tag;
158       rev_tag = false;
159     }
160   }
161   if (!a || !b) return a ? a : b;
162   if (a->pri < b->pri) {
163     a->push();
164     a->r = merge(a->r, b);
165     a->pull();
166     return a;
167   } else {
168     b->push();
169     b->l = merge(a, b->l);
170     b->pull();
171     return b;
172   }
173 }
174
175 void splsz(Node* T, Node*& a, Node*& b, ll s
176 ) {
177   if (!T) {
178     a = b = nullptr;
179     return;
180   }
181   T->push();
182   ll lsr = T->l ? T->l->sz : 0;
183   if (lsr + 1 <= s) {
184     a = T;
185     splsz(T->r, a->r, b, s - (lsr + 1));
186     a->pull();
187   } else {
188     b = T;
189     splsz(T->l, a, b->l, s);
190     b->pull();
191   }
192 }
193
194 void rev_range(Node*& root, ll l, ll r) {
195   // 1-indexed
196   Node* left,* mid,* right;
197   splsz(root, left, mid, l - 1);
198   splsz(mid, mid, right, r - l + 1);
199   mid->push(); // this line isn't
200   necessary
201   if (mid) mid->rev_tag = !mid->rev_tag;
202   root = merge(left, mid);
203   root = merge(root, right);
204 }
205
206 void inorder(Node* root, string& res) {
207   if (!root) return;
208   root->push();
209   inorder(root->l, res);
210   res += char(root->val);
211   inorder(root->r, res);
212 }
213
214 int main() {
215   ios::sync_with_stdio(false);
216   cin.tie(nullptr);
217
218   ll n, q;
219   cin >> n >> q;
220
221   string s;
222   cin >> s;
223
224   Node* root = nullptr;
225   for (ll i = 0; i < n; ++i) {
226     root = merge(root, new Node(s[i]));
227   }
228
229   rep (q) {
230     ll l, r;
231     cin >> l >> r;
232     rev_range(root, l, r);
233   }
234
235   string res = "";
236   inorder(root, res);
237   cout << res << '\n';
238
239   return 0;
240 }
241
242 // CSES Reversals and Sums
243 // Given an array of n integers, you have to
244 // process following operations:
245 // reverse a subarray
246 // calculate the sum of values in a subarray
247
248 #include <bits/stdc++.h>
249 using namespace std;
250 using ll = long long;
251 using pll = pair<ll, ll>;
252 template<class T> using vec = vector<T>;
253 #define endl '\n'
254 #define ff first
255 #define ss second
256 #define pb push_back
257 #define rep(n) for (ll _ = 1; _ <= n; ++_)
258
259 // -----
260
261 mt19937 rng(chrono::steady_clock::now().
262   time_since_epoch().count());
263
264 class Node {
265 public:
266   Node* l,* r;
267   ll val, pri, sz;
268   ll sum;
269   bool rev_tag;
270   Node(ll _val) {
271     l = r = nullptr;
272     val = _val;
273     pri = rng();
274     sz = 1;
275     sum = _val;
276     rev_tag = false;
277   }
278   void pull() {
279     sz = 1;
280     if (l) sz += l->sz;
281     if (r) sz += r->sz;
282     sum = val;
283     if (l) sum += l->sum;
284     if (r) sum += r->sum;
285   }
286   void push() {
287     if (rev_tag) {
288       swap(l, r);
289       if (l) l->rev_tag = !l->rev_tag;
290       if (r) r->rev_tag = !r->rev_tag;
291       rev_tag = false;
292     }
293   }
294   Node* merge(Node* a, Node* b) {
295     if (!a || !b) return a ? a : b;
296     if (a->pri < b->pri) {
297       a->push();
298       a->r = merge(a->r, b);
299       a->pull();
300       return a;
301     } else {
302       b->push();
303       b->l = merge(a, b->l);
304       b->pull();
305       return b;
306     }
307   }
308 }
309
310 void splsz(Node* root, Node*& a, Node*& b,
311   ll s) {
312   if (!root) {
313     a = b = nullptr;
314     return;
315   }
316   root->push();
317   ll lsr = root->l ? root->l->sz : 0;
318   if (lsr + 1 <= s) {
319     a = root;
320     splsz(root->r, a->r, b, s - (lsr +
321       1));
322     a->pull();
323   } else {
324     b = root;
325     splsz(root->l, a, b->l, s);
326     b->pull();
327   }
328 }
329
330 void rev_range(Node*& root, ll l, ll r) {
331   // 1-indexed
332   Node* left,* mid,* right;
333   splsz(root, left, mid, l - 1);
334   splsz(mid, mid, right, r - l + 1);
335   if (mid) mid->rev_tag = !mid->rev_tag;
336   root = merge(left, mid);
337   root = merge(root, right);
338 }
339
340 ll qry_range_sum(Node*& root, ll l, ll r) {
341   Node* left,* mid,* right;
342   splsz(root, left, mid, l - 1);
343   splsz(mid, mid, right, r - l + 1);
344   ll res = 0;
345   if (mid) res = mid->sum;
346   root = merge(left, mid);
347   root = merge(root, right);

```

```

345     return res;
346 }
347
348 int main() {
349     ios::sync_with_stdio(false);
350     cin.tie(nullptr);
351
352     ll n, q;
353     cin >> n >> q;
354
355     Node* root = nullptr;
356     for (ll i = 0; i < n; ++i) {
357         ll val; cin >> val;
358         root = merge(root, new Node(val));
359     }
360
361     rep(q) {
362         ll act, l, r;
363         cin >> act >> l >> r;
364         if (act == 1) {
365             rev_range(root, l, r);
366         } else if (act == 2) {
367             cout << qry_range_sum(root, l, r) << endl;
368         }
369     }
370
371     return 0;
372 }
373
374 // =====
375 // =====
376
377 // SPOJ ORDERSET
378 // In this problem, you have to maintain a
379 // dynamic set of numbers
380 // which support the two fundamental
381 // operations
382 // INSERT(S,x): if x is not in S, insert
383 // x into S
384 // DELETE(S,x): if x is in S, delete x
385 // from S
386 // and the two type of queries
387 // K-TH(S) : return the k-th smallest
388 // element of S
389 // COUNT(S,x): return the number of
390 // elements of S smaller than x
391
392 #include <bits/stdc++.h>
393 using namespace std;
394 using ll = long long;
395 using pll = pair<ll, ll>;
396 template<class T> using vec = vector<T>;
397 #define endl '\n'
398 #define ff first
399 #define ss second
400 #define pb push_back
401 #define rep(n) for (ll _ = 1; _ <= n; ++_)
402
403 -----
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557

```

4.5 Sparse Table

```

1 // if the max size of arr is 200000
2 vector<ll> arr;
3
4 // -----
5
6 // Sparse Table
7 const ll lgmx = 17; // floor(Log2(200000))
8 ll rmq[lgmx + 1][200000 + 1];
9 void init(ll n) { // O(nlogn)
10     for (ll i = 1; i <= n; ++i) rmq[0][i] =
11         arr[i];
12     for (ll i = 1; i <= lgmx; ++i)
13         for (ll j = 1; j + (1 << i) - 1 <= n; ++j)
14             rmq[i][j] = min(rmq[i - 1][j],
15                             rmq[i - 1][j + (1 << (i - 1))]);
16 }
17 ll flg(ull x) {return 63 - __builtin_clzll(x);}
18 ll query(ll l, ll r) { // O(1)
19     ll h = flg(r - l + 1);

```

```

18     return min(rmq[h][l], rmq[h][r - (1 << h
19         ) + 1]);
20 }
21 // -----
22 // initialize the array
23 ll n; cin >> n;
24 arr.resize(n + 1);
25 for (ll i = 1; i <= n; ++i) cin >> arr[i];
26 // initialize the sparse table
27 init(n);

```

5 Geometry

5.1 Convex Hull

```

1 // pts = {p0, p1, ... pn-1}, 0-based
2 // the points in pts should be distinct
3 vector<vec> convexHull(const vector<vec> &
4     _pts) {
5     vector<vec> pts = _pts;
6     sort(pts.begin(), pts.end());
7
8     ll n = pts.size();
9     vector<vec> hull(1, pts[0]);
10    for (ll i = 1; i < n; ++i) {
11        while (hull.size() >= 2 &&
12            ori(hull[hull.size() - 2],
13                hull.back(), pts[i]) < 0)
14            hull.pop_back();
15        hull.push_back(pts[i]);
16    }
17
18    ll m = hull.size();
19    for (ll i = n - 2; i >= 0; --i) {
20        while (hull.size() - m + 1 >= 2 &&
21            ori(hull[hull.size() - 2],
22                hull.back(), pts[i]) < 0)
23            hull.pop_back();
24        hull.push_back(pts[i]);
25    }
26    hull.pop_back();
27
28    return hull;

```

5.2 Vector

```

1 class vec {
2 public:
3     ll x, y;
4     vec() {}

```

```

5     vec(ll _x, ll _y) : x(_x), y(_y) {}
6     vec operator+(const vec& v) const {
7         return vec(this->x + v.x, this->y +
8             v.y);
9     }
10    vec operator-(const vec& v) const {
11        return vec(this->x - v.x, this->y -
12            v.y);
13    }
14    ll operator*(const vec& v) const {
15        return this->x * v.x + this->y * v.y;
16    }
17    ll operator^(const vec& v) const {
18        return this->x * v.y - this->y * v.x;
19    }
20    bool operator<(const vec& v) const {
21        if (this->x != v.x) return this->x <
22            v.x;
23        return this->y < v.y;
24    }
25    vec& operator=(const vec& v) {
26        this->x = v.x;
27        this->y = v.y;
28        return *this;
29    }
30
31    ll sign(ll x) {
32        if (x == 0) return 0;
33        if (x < 0) return -1;
34        return 1;
35    }
36    ll ori(const vec& o, const vec& a, const vec&
37        b) {
38        return sign((a - o) ^ (b - o));
39    }
40    bool isCollinear(const vec& a, const vec& b,
41        const vec& c) {
42        return ori(a, b, c) == 0;
43    }
44    bool isOnSeg(const vec& a, const vec& b,
45        const vec& p) {
46        return isCollinear(a, b, p) && sign((p -
47            a) * (p - b)) <= 0;
48    }

```

5.3 Simple Polygon Area

```

1 // pts = {p0, p1, ... pn-1, p0}, 0-based
2 // for simple polygon area
3 ll shoelace2(const vector<vec> &pts) {
4     ll n = pts.size() - 2, res = 0;
5     for (ll i = 0; i <= n; ++i) {
6         res += pts[i].x * pts[i + 1].y;
7         res -= pts[i].y * pts[i + 1].x;
8     }
9     return abs(res);
10 }

```

5.4 Line Segment Intersection Test

```

1 bool isSegInter(const vec& a, const vec& b,
2     const vec& c, const vec& d) {
3     ll ori1 = ori(a, b, c);
4     ll ori2 = ori(a, b, d);
5     ll ori3 = ori(c, d, a);
6     ll ori4 = ori(c, d, b);
7     if (isCollinear(a, b, c) && isCollinear(
8         a, b, d)) {
9         return isOnSeg(a, b, c) || isOnSeg(a
10             , b, d) ||
11             isOnSeg(c, d, a) || isOnSeg(c
12                 , d, b);
13     }
14     return ori1 * ori2 <= 0 && ori3 * ori4
15         <= 0;

```

6 Graph

6.1 Kosaraju

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 #define pb push_back
5 #define rep(n) for (ll _ = 1; _ <= n; ++_)
6
7 ll V, E;
8
9 stack<ll> stk;
10 vector<bool> vis;
11 vector<vector<ll>> adj;
12 void dfs1(ll u) {
13     vis[u] = true;
14     for (ll v : adj[u]) {
15         if (vis[v]) continue;
16         dfs1(v);
17     }
18     stk.push(u);
19 }
20
21 ll sccCnt;
22 vector<ll> scc;
23 vector<vector<ll>> radj;
24 void dfs2(ll u, ll sccIdx) {
25     scc[u] = sccIdx;
26     for (ll v : radj[u]) {
27         if (scc[v] != -1) continue;
28         dfs2(v, sccIdx);
29     }
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr);
35
36     cin >> V >> E;
37     vis.resize(V + 1, false);

```

```

38     adj.resize(V + 1);
39     radj.resize(V + 1);
40     rep(E) {
41         ll x, y; cin >> x >> y;
42         adj[x].pb(y);
43         radj[y].pb(x);
44     }
45
46     for (ll u = 1; u <= V; ++u) {
47         if (vis[u]) continue;
48         dfs1(u);
49     }
50
51     sccCnt = 0;
52     scc.resize(V + 1, -1);
53     while (not stk.empty()) {
54         ll u = stk.top(); stk.pop();
55         if (scc[u] != -1) continue;
56         dfs2(u, ++sccCnt);
57     }
58
59     cout << sccCnt << '\n';
60     for (ll i = 1; i <= V; ++i) {
61         cout << scc[i] << ' ';
62     } cout << '\n';
63
64     return 0;
65 }

```

6.2 Dinic Min Cut

```

1 // 最小割是選一些邊可以阻斷所有水流且這些邊
2 // 的容量和最小
3 // 輸出最小割的容量和選到的邊
4 // (最大流量和 == 最小割容量和)
5
6 #include <bits/stdc++.h>
7 using namespace std;
8 using ll = long long;
9 #define pb push_back
10 constexpr ll INF = 1e18;
11
12 struct Node {
13     bool isPseudo;
14     ll nd, pvIdx, cap;
15     Node(ll _nd, ll _pvIdx, ll _cap, bool
16         _isPseudo) : nd(_nd), pvIdx(_pvIdx),
17         cap(_cap), isPseudo(_isPseudo) {}
18 };
19
20 ll n, m, st, en;
21 vector<ll> level, iter;
22 vector<vector<Node>> adj;
23
24 void bfs() {
25     level.clear(); level.resize(n + 1, -1);
26     level[st] = 0;
27
28     queue<ll> q; q.push(st);
29     while (not q.empty()) {
30         ll sz = q.size();
31         for (ll _ = 1; _ <= sz; ++_) {
32             ll u = q.front(); q.pop();

```



```

30     for (const Node& v : adj[u]) {
31         if (v.cap > 0 and level[v.nd]
32             ] == -1) {
33             level[v.nd] = level[u] +
34                 1;
35             q.push(v.nd);
36         }
37     }
38 }
39
40 ll dfs(ll u, ll flow) {
41     if (u == en) return flow;
42     for (ll& i = iter[u]; i < adj[u].size();
43         ++i) {
44         Node& v = adj[u][i];
45         if (v.cap > 0 and level[v.nd] ==
46             level[u] + 1) {
47             ll fw = dfs(v.nd, min(flow, v.
48                 cap));
49             if (fw > 0) {
50                 v.cap -= fw;
51                 adj[v.nd][v.pvIdx].cap += fw
52                 ;
53                 return fw;
54             }
55         }
56     }
57     return 0; // for compile warning
58 }
59
60 ll maxFlow() {
61     ll res = 0;
62     while (true) {
63         bfs();
64         if (level[en] == -1) break;
65
66         ll cur;
67         iter.clear(); iter.resize(n + 1);
68         while ((cur = dfs(st, INF)) > 0) {
69             res += cur;
70         }
71     }
72     return res;
73 }
74
75 vector<bool> vis;
76 void markSetS(ll u) {
77     vis[u] = true;
78     for (const Node& v : adj[u]) {
79         if (v.isPseudo) continue;
80         if (not vis[v.nd] and v.cap > 0) {
81             markSetS(v.nd);
82         }
83     }
84 }
85
86 void outputCut() {
87     for (ll u = 1; u <= n; ++u) {
88         if (not vis[u]) continue; // set T
89         for (const Node& v : adj[u]) {
90             if (v.isPseudo) continue;
91             if (not vis[v.nd]) {
92                 cout << u << ' ' << v.nd <<
93                     '\n';

```

```

89     }
90     }
91 }
92 }
93
94 int main() {
95     ios::sync_with_stdio(false);
96     cin.tie(nullptr);
97
98     cin >> n >> m;
99     adj.clear(); adj.resize(n + 1);
100     for (ll i = 1; i <= m; ++i) {
101         ll u, v; cin >> u >> v;
102
103         adj[u].pb(Node(v, adj[v].size(), 1,
104             false));
105         adj[v].pb(Node(u, adj[u].size() - 1,
106             0, true));
107
108         adj[v].pb(Node(u, adj[u].size(), 1,
109             false));
110         adj[u].pb(Node(v, adj[v].size() - 1,
111             0, true));
112     }
113     st = 1;
114     en = n;
115     cout << maxFlow() << '\n'; // maxFlow
116         == minCut
117
118     vis.resize(n + 1);
119     markSetS(st);
120     outputCut();
121
122     return 0;
123 }

```

6.3 AP

```

1 void dfs(ll u, ll pa = -1) {
2     ll ch = 0; // 紀錄子節點樹
3     low[u] = in[u] = ++t;
4     for (ll v : adj[u]) {
5         if (v == pa) continue;
6         if (in[v] == -1) {
7             ch += 1; // 子節點數加一
8             dfs(v, u);
9             low[u] = min(low[u], low[v]);
10            if (pa != -1 and low[v] >= in[u]
11                ]) "u is AP" // find AP
12        } else if (in[v] < in[u]) {
13            low[u] = min(low[u], in[v]);
14        }
15    }
16    if (pa == -1 and ch >= 2) "u is AP" //
        root 額外判斷

```

6.4 Dinic

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 #define pb push_back
5 constexpr ll INF = 1e18;
6
7 struct Node {
8     ll nd, pvIdx, cap;
9     Node(ll _nd, ll _pvIdx, ll _cap) : nd(
10         _nd), pvIdx(_pvIdx), cap(_cap) {}
11 };
12
13 ll n, m, st, en;
14 vector<ll> level, iter;
15 vector<vector<Node>> adj;
16
17 void bfs() {
18     level.clear(); level.resize(n + 1, -1);
19     level[st] = 0;
20
21     queue<ll> q; q.push(st);
22     while (not q.empty()) {
23         ll sz = q.size();
24         for (ll _ = 1; _ <= sz; ++_) {
25             ll u = q.front(); q.pop();
26             for (const Node& v : adj[u]) {
27                 if (v.cap > 0 and level[v.nd]
28                     ] == -1) {
29                     level[v.nd] = level[u] +
30                         1;
31                     q.push(v.nd);
32                 }
33             }
34         }
35     }
36 }
37
38 ll dfs(ll u, ll flow) {
39     if (u == en) return flow;
40     for (ll& i = iter[u]; i < adj[u].size();
41         ++i) {
42         Node& v = adj[u][i];
43         if (v.cap > 0 and level[v.nd] ==
44             level[u] + 1) {
45             ll fw = dfs(v.nd, min(flow, v.
46                 cap));
47             if (fw > 0) {
48                 v.cap -= fw;
49                 adj[v.nd][v.pvIdx].cap += fw
50                 ;
51                 return fw;
52             }
53         }
54     }
55     return 0; // for compile warning
56 }
57
58 ll maxFlow() {
59     ll res = 0;
60     while (true) {
61         bfs();
62         if (level[en] == -1) break;
63
64         ll cur;
65         iter.clear(); iter.resize(n + 1);
66         while ((cur = dfs(st, INF)) > 0) {

```

```

60         res += cur;
61     }
62 }
63
64 return res;
65 }
66
67 int main() {
68     ios::sync_with_stdio(false);
69     cin.tie(nullptr);
70
71     cin >> n >> m;
72     adj.clear(); adj.resize(n + 1);
73     for (ll i = 1; i <= m; ++i) {
74         ll u, v, cap; cin >> u >> v >> cap;
75         adj[u].pb(Node(v, adj[v].size(), cap
76             ));
77         adj[v].pb(Node(u, adj[u].size() - 1,
78             0));
79     }
80     st = 1;
81     en = n;
82     cout << maxFlow() << '\n';
83
84     return 0;
85 }

```

6.5 Dijkstra

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     const ll INF = 1e18;
6
7     ll n, m; cin >> n >> m;
8     vector<vector<pll>> adj(n + 1);
9     rep (m) {
10         ll u, v, w; cin >> u >> v >> w;
11         adj[u].pb(pll(w, v));
12     }
13
14     vector<ll> dis(n + 1, INF);
15     priority_queue<pll, vector<pll>, greater<
16         pll>> pq;
17
18     dis[1] = 0;
19     pq.push(pll(0, 1));
20     while (not pq.empty()) {
21         auto [uw, u] = pq.top(); pq.pop();
22
23         if (uw > dis[u]) continue;
24
25         for (auto [w, v] : adj[u]) {
26             if (dis[u] + w < dis[v]) {
27                 dis[v] = dis[u] + w;
28                 pq.push(pll(dis[v], v));
29             }
30         }
31     }

```

6.6 Tarjan 縮點

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 ll n, m, t = 0, sccCnt = 0, sccIdx = 0;
8 stack<ll> stk;
9 vector<bool> inStack;
10 vector<ll> in, low, scc, val;
11 vector<vector<ll>> adj;
12 void dfs(ll u) {
13     low[u] = in[u] = ++t;
14     stk.push(u);
15     inStack[u] = true;
16     for (ll v : adj[u]) {
17         if (in[v] == -1) {
18             dfs(v);
19             low[u] = min(low[u], low[v]);
20         } else if (inStack[v]) {
21             low[u] = min(low[u], low[v]);
22         }
23     }
24     if (in[u] == low[u]) {
25         sccCnt += 1;
26         sccIdx += 1;
27         ll cur;
28         do {
29             cur = stk.top(); stk.pop();
30             inStack[cur] = false;
31             scc[cur] = sccIdx;
32         } while (cur != u);
33     }
34 }
35
36 vector<bool> vis2;
37 vector<ll> val2, dp2;
38 vector<vector<ll>> adj2;
39 void dfs2(ll u) {
40     vis2[u] = true;
41     for (ll v : adj2[u]) {
42         if (not vis2[v]) dfs2(v);
43         dp2[u] = max(dp2[u], val2[u] + dp2[v]);
44     }
45 }
46
47 int main() {
48     ios::sync_with_stdio(false);
49     cin.tie(nullptr);
50
51     cin >> n >> m;
52     inStack.resize(n + 1);
53     in.resize(n + 1, -1);
54     low.resize(n + 1);
55     scc.resize(n + 1);
56     val.resize(n + 1);
57     adj.resize(n + 1);
58     for (ll u = 1; u <= n; ++u) cin >> val[u];
59     for (ll i = 1; i <= m; ++i) {
60         ll u, v; cin >> u >> v;
61         adj[u].pb(v);

```

```

62     }
63     for (ll u = 1; u <= n; ++u) {
64         if (in[u] != -1) continue;
65         dfs(u);
66     }
67     vis2.resize(sccCnt + 1);
68     val2.resize(sccCnt + 1);
69     dp2.resize(sccCnt + 1);
70     adj2.resize(sccCnt + 1);
71     for (ll u = 1; u <= n; ++u) {
72         for (ll v : adj[u]) {
73             val2[scc[u]] += val[u];
74             for (ll v : adj[u]) {
75                 if (scc[v] == scc[u]) continue;
76                 adj2[scc[u]].pb(scc[v]);
77             }
78         }
79     }
80     for (ll u = 1; u <= sccCnt; ++u) {
81         dp2[u] = val2[u];
82     }
83     for (ll u = 1; u <= sccCnt; ++u) {
84         if (vis2[u]) continue;
85         dfs2(u);
86     }
87
88     cout << *max_element(dp2.begin() + 1,
89                          dp2.end()) << '\n';
90
91     return 0;
92 }

```

6.7 Dinic 無權二分圖最大匹配

```

1 // 輸出無權二分圖最大匹配的邊數量和選到的邊
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 using ll = long long;
6 #define pb push_back
7 constexpr ll INF = 1e18;
8
9 struct Node {
10     ll nd, pvIdx, cap;
11     Node(ll _nd, ll _pvIdx, ll _cap) : nd(_nd), pvIdx(_pvIdx), cap(_cap) {}
12 };
13
14 ll n1, n2, n, m, st, en;
15 vector<ll> level, iter;
16 vector<vector<Node>> adj;
17
18 void bfs() {
19     level.clear(); level.resize(n + 1, -1);
20     level[st] = 0;
21
22     queue<ll> q; q.push(st);
23     while (not q.empty()) {
24         ll sz = q.size();
25         for (ll _ = 1; _ <= sz; ++_) {
26             ll u = q.front(); q.pop();
27             for (const Node& v : adj[u]) {

```

```

28                 if (v.cap > 0 and level[v.nd] == -1) {
29                     level[v.nd] = level[u] + 1;
30                     q.push(v.nd);
31                 }
32             }
33         }
34     }
35
36     ll dfs(ll u, ll flow) {
37         if (u == en) return flow;
38         for (ll& i = iter[u]; i < adj[u].size(); ++i) {
39             Node& v = adj[u][i];
40             if (v.cap > 0 and level[v.nd] == level[u] + 1) {
41                 ll fw = dfs(v.nd, min(flow, v.cap));
42                 if (fw > 0) {
43                     v.cap -= fw;
44                     adj[v.nd][v.pvIdx].cap += fw;
45                     return fw;
46                 }
47             }
48         }
49         return 0; // for compile warning
50     }
51
52     ll maxFlow() {
53         ll res = 0;
54         while (true) {
55             bfs();
56             if (level[en] == -1) break;
57
58             ll cur;
59             iter.clear(); iter.resize(n + 1);
60             while ((cur = dfs(st, INF)) > 0) {
61                 res += cur;
62             }
63         }
64         return res;
65     }
66 }
67
68 int main() {
69     ios::sync_with_stdio(false);
70     cin.tie(nullptr);
71
72     cin >> n1 >> n2 >> m;
73     n = n1 + n2 + 2;
74     st = n1 + n2 + 1;
75     en = n1 + n2 + 2;
76     adj.clear(); adj.resize(n + 1);
77     for (ll i = 1; i <= m; ++i) {
78         ll u, v; cin >> u >> v; v += n1;
79         adj[u].pb(Node(v, adj[v].size() - 1, 1));
80         adj[v].pb(Node(u, adj[u].size() - 1, 1));
81     }
82     m += n1 + n2;
83     for (ll i = 1; i <= n1; ++i) {
84         adj[st].pb(Node(i, adj[i].size(), 1));

```

```

85         adj[i].pb(Node(st, adj[st].size() - 1, 0));
86     }
87     for (ll i = n1 + 1; i <= n1 + n2; ++i) {
88         adj[i].pb(Node(en, adj[en].size(), 1));
89         adj[en].pb(Node(i, adj[i].size() - 1, 0));
90     }
91     cout << maxFlow() << '\n';
92     for (ll u = 1; u <= n1; ++u) {
93         for (const Node& v : adj[u]) {
94             if (v.nd != st and v.cap == 0) {
95                 cout << u << ' ' << v.nd << '\n';
96             }
97         }
98     }
99     return 0;
100 }
101 }

```

6.8 Floyd Warshall

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     cin >> n >> m;
6     mtx.clear(); mtx.resize(n + 1, vector<ll>(n + 1, INF));
7     for (ll i = 1; i <= n; ++i) mtx[i][i] = 0;
8     rep (m) {
9         ll x, y, w; cin >> x >> y >> w;
10         mtx[x][y] = min(mtx[x][y], w);
11         mtx[y][x] = min(mtx[y][x], w);
12     } // "min" is used to prevent the multiple edges
13
14     for (ll k = 1; k <= n; ++k) {
15         for (ll i = 1; i <= n; ++i) {
16             for (ll j = 1; j <= n; ++j) {
17                 mtx[i][j] = min(mtx[i][j], mtx[i][k] + mtx[k][j]);
18             }
19         }
20     }
21 }

```

6.9 MST Prim

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     constexpr ll INF = 1e18;
6
7     ll n, m; cin >> n >> m;
8     vector<vector<pll>> adj(n + 1);
9     rep (m) {

```



```

10     ll u, v, w; cin >> u >> v >> w;
11     adj[u].pb(p11(w, v));
12     adj[v].pb(p11(w, u));
13 }
14
15 priority_queue<p11, vector<p11>, greater
16 <p11>> pq;
17 vector<bool> vis(n + 1, false);
18 vector<ll> curw(n + 1, INF);
19
20 pq.push(p11(0, 1));
21 curw[1] = 0;
22
23 ll mstCost = 0, mstEdgesCnt = 0;
24 while (mstEdgesCnt < n - 1) {
25     if (pq.empty()) {
26         // the graph is disconnected (
27         // MST D.N.E.)
28         return 0;
29     }
30     auto [uw, u] = pq.top(); pq.pop();
31
32     if (uw > curw[u]) continue;
33
34     vis[u] = true;
35     mstCost += uw;
36     mstEdgesCnt += (u == 1 ? 0 : 1);
37
38     for (const auto& [w, v] : adj[u]) {
39         if (not vis[v] and w < curw[v]) {
40             pq.push(p11(w, v));
41             curw[v] = w;
42         }
43     }
44 }
45 return 0;
46 }

```

6.10 MST Kruskal

```

1 vector<ll> djs, sz;
2 ll find(ll u) {...}
3 void join(ll u, ll v) {...}
4 void init(ll n) {...}
5
6 class Edge{
7 public:
8     ll u, v, w;
9 };
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     // nodes are 1-indexed
16     // edges are 0-indexed
17
18     ll n, m; cin >> n >> m;
19     vector<Edge> edges(m);
20     for (auto& [u, v, w] : edges) {

```

```

21     cin >> u >> v >> w;
22 }
23 sort(all(edges), [](const Edge& e1,
24     const Edge e2) -> bool {
25     return e1.w < e2.w;
26 });
27 init(n);
28
29 ll mstCost = 0, mstEdgesCnt = 0;
30 for (const auto& [u, v, w] : edges) {
31     if (find(u) == find(v)) continue;
32     join(u, v);
33     mstCost += w;
34     mstEdgesCnt += 1;
35     if (mstEdgesCnt == n - 1) break;
36 }
37
38 // if (mstEdgesCnt < n - 1) // the graph
39 // is disconnected (MST D.N.E.)
40
41 return 0;

```

6.11 Topo-sort DFS

```

1 ll v;
2 vector<ll> topo;
3 vector<bool> vis;
4 vector<vector<ll>> adj(v + 1);
5 void dfs(ll node) {
6     vis[node] = true;
7     for (ll neighbor : adj[node]) {
8         if (vis[neighbor]) continue;
9         dfs(neighbor);
10    }
11    topo.pb(node);
12 }
13
14 reverse(topo.begin(), topo.end());

```

6.12 Topo-sort Kahn

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     ll n, m; cin >> n >> m;
6     vector<ll> indeg(n + 1, 0);
7     vector<vector<ll>> adj(n + 1);
8     rep (m) {
9         ll u, v; cin >> u >> v;
10        adj[u].pb(v);
11        indeg[v] += 1;
12    }
13
14    vector<ll> res; res.reserve(n);
15    queue<ll> q; // you can use any pool data
16    // structure here
17    for (ll u = 1; u <= n; ++u) {
18        if (indeg[u] == 0) {

```

```

18        q.push(u);
19        res.pb(u);
20    }
21    while (not q.empty()) {
22        ll u = q.front(); q.pop();
23        for (ll v : adj[u]) {
24            indeg[v] -= 1;
25            if (indeg[v] == 0) {
26                q.push(v);
27                res.pb(v);
28            }
29        }
30    }
31
32    // if (res.size() < n) // there exists a
33    // directed cycle
34
35    return 0;
36 }

```

6.13 Tarjan 2-SAT

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using p11 = pair<ll, ll>;
5 #define pb push_back
6
7 ll n, t = 0, sccCnt = 0, sccIdx = 0;
8 stack<ll> stk;
9 vector<bool> inStack;
10 vector<ll> in, low, scc;
11 vector<vector<ll>> adj;
12 void dfs(ll u) {
13     low[u] = in[u] = ++t;
14     stk.push(u);
15     inStack[u] = true;
16     for (ll v : adj[u]) {
17         if (in[v] == -1) {
18             dfs(v);
19             low[u] = min(low[u], low[v]);
20         } else if (inStack[v]) {
21             low[u] = min(low[u], low[v]);
22         }
23     }
24     if (in[u] == low[u]) {
25         sccCnt += 1;
26         sccIdx += 1;
27         ll cur;
28         do {
29             cur = stk.top(); stk.pop();
30             inStack[cur] = false;
31             scc[cur] = sccIdx;
32         } while (cur != u);
33     }
34 }
35
36 int main() {
37     ios::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     ll x, y; cin >> x >> y;

```

```

41     n = 2*y;
42     inStack.resize(n + 1);
43     in.resize(n + 1, -1);
44     low.resize(n + 1);
45     scc.resize(n + 1);
46     adj.resize(n + 1);
47     for (ll i = 1; i <= x; ++i) {
48         char c1, c2;
49         ll a, b;
50         cin >> c1 >> a >> c2 >> b;
51
52         if (c1 == '+') {
53             adj[a + y].pb((c2 == '+' ? b : b +
54                 + y));
55         } else {
56             adj[a].pb((c2 == '+' ? b : b + y
57                 ));
58         }
59
60         if (c2 == '+') {
61             adj[b + y].pb((c1 == '+' ? a : a +
62                 + y));
63         } else {
64             adj[b].pb((c1 == '+' ? a : a + y
65                 ));
66         }
67     }
68
69     for (ll u = 1; u <= n; ++u) {
70         if (in[u] != -1) continue;
71         dfs(u);
72     }
73
74     bool valid = true;
75     string s = "";
76     for (ll u = 1; u <= y; ++u) {
77         if (scc[u] == scc[u + y]) {
78             valid = false;
79             break;
80         } else if (scc[u] < scc[u + y]) {
81             s += "+ ";
82         } else {
83             s += "- ";
84         }
85     }
86
87     cout << (valid ? s : "IMPOSSIBLE") << '\n';
88
89     return 0;
90 }

```

6.14 Bridge

```

1 void dfs(ll u, ll pa = -1) {
2     low[u] = in[u] = ++t;
3     for (ll v : adj[u]) {
4         if (v == pa) continue;
5         if (in[v] == -1) {
6             dfs(v, u);
7             low[u] = min(low[u], low[v]);
8             if (low[v] > in[u]) "edge (u, v)
9                 is bridge" // find bridge
10         } else if (in[v] < in[u]) {

```

```

10         low[u] = min(low[u], in[v]);
11     }
12 }
13 }

```

6.15 Bellman Ford Detects Negative Cycle

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define ff first
6 #define ss second
7 #define pb push_back
8 #define rep(n) for (ll _ = 1; _ <= n; ++_)
9
10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     const ll INF = 5e12 + 1000;
15
16     ll n, m; cin >> n >> m;
17     vector<ll> dis(n + 1, INF);
18     vector<vector<pll>> adj(n + 1);
19     rep(m) {
20         ll u, v, w; cin >> u >> v >> w;
21         adj[u].pb(pll(w, v));
22     }
23     for (ll i = 1; i <= n; ++i) {
24         adj[0].pb(pll(0, i));
25     }
26
27     dis[0] = 0;
28     rep(n - 1) {
29         bool updated = false;
30         for (ll u = 0; u <= n; ++u) {
31             for (const auto& [w, v] : adj[u]) {
32                 if (dis[u] < INF and dis[u] + w <
33                     dis[v]) {
34                     dis[v] = dis[u] + w;
35                     updated = true;
36                 }
37             }
38             if (not updated) break;
39         }
40
41         bool hasNegativeCycle = false;
42         for (ll u = 0; not hasNegativeCycle and u
43             <= n; ++u) {
44             for (const auto& [w, v] : adj[u]) {
45                 if (dis[u] < INF and dis[u] + w < dis[
46                     v]) {
47                     hasNegativeCycle = true;
48                     break;
49                 }
50             }
51         }
52         return 0;
53     }

```

6.16 Contracting Vertices Based on Specific Conditions

```

1 // define a DSU first
2
3 ll v, e; cin >> v >> e; // the number of
4 // vertices and edges
5
6 vector<pll> edges;
7 rep(e) {
8     ll x, y; cin >> x >> y; // bidirectional
9     // edge (x, y)
10     if (fit_some_condition) join(x, y);
11     else edges.pb(pll(x, y));
12 }
13
14 vector<vector<ll>> adj; // undirected graph
15 for (const pll& edge : edges) {
16     ll x = find_root(edge.ff), y = find_root(
17         edge.ss);
18     adj[x].pb(y);
19     adj[y].pb(x);
20 }

```

6.17 Tarjan

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 ll n, m, t = 0, sccCnt = 0, sccIdx = 0;
8 stack<ll> stk;
9 vector<bool> inStack;
10 vector<ll> in, low, scc;
11 vector<vector<ll>> adj;
12 void dfs(ll u) {
13     low[u] = in[u] = ++t;
14     stk.push(u);
15     inStack[u] = true;
16     for (ll v : adj[u]) {
17         if (in[v] == -1) {
18             dfs(v);
19             low[u] = min(low[u], low[v]);
20         } else if (inStack[v]) {
21             low[u] = min(low[u], low[v]);
22         }
23     }
24     if (in[u] == low[u]) {
25         sccCnt += 1;
26         sccIdx += 1;
27         ll cur;
28         do {
29             cur = stk.top(); stk.pop();
30             inStack[cur] = false;
31             scc[cur] = sccIdx;

```

```

32     } while (cur != u);
33 }
34 }
35
36 int main() {
37     ios::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     cin >> n >> m;
41     inStack.resize(n + 1);
42     in.resize(n + 1, -1);
43     low.resize(n + 1);
44     scc.resize(n + 1);
45     adj.resize(n + 1);
46     for (ll i = 1; i <= m; ++i) {
47         ll u, v; cin >> u >> v;
48         adj[u].pb(v);
49     }
50
51     for (ll u = 1; u <= n; ++u) {
52         if (in[u] != -1) continue;
53         dfs(u);
54     }
55
56     cout << sccCnt << '\n';
57     for (ll u = 1; u <= n; ++u) {
58         cout << scc[u] << ' ';
59     } cout << '\n';
60
61     return 0;
62 }

```

7 Math

7.1 Big Integer Addition and Multiplication

```

1 vector<int> strToVec(string str, int sz) {
2     vector<int> r(sz, 0);
3     int len = str.length();
4     for (int i = strLength - 1, idx = 0; i
5         >= 0; --i, ++idx) {
6         r[idx] = str[i] - '0';
7     }
8     return r;
9 }
10 // for example:
11 // strToVec("677", 4) -> 7 7 6 0
12 // strToVec("8829", 4) -> 9 2 8 8
13
14 // addition
15 string add(string x, string y) {
16     ll n = max(x.length(), y.length());
17     vector<ll> xdigit = strToVec(x, n + 1);
18     vector<ll> ydigit = strToVec(y, n + 1);
19     vector<ll> result(n + 1, 0);
20     ll carry = 0;
21     for (ll i = 0; i < n + 1; ++i) {
22         result[i] = xdigit[i] + ydigit[i] +
23             carry;

```

```

22         if (result[i] >= 10) {
23             result[i] %= 10;
24             carry = 1;
25         }
26         else carry = 0;
27     }
28     ll start;
29     for (ll i = n; i >= 0; --i) {
30         if (result[i] != 0) {
31             start = i;
32             break;
33         }
34     }
35     string r = "";
36     for (ll i = start; i >= 0; --i) {
37         r += result[i] + '0';
38     }
39     return r;
40 }
41
42 // multiplication
43 string product(string x, string y) {
44     ll xlength = x.length();
45     ll ylength = y.length();
46     ll n = max(xlength, ylength);
47     vector<ll> xdigit = strToVec(x, xlength);
48     vector<ll> ydigit = strToVec(y, ylength);
49     vector<ll> result(2*n, 0);
50     for (ll i = 0; i < xlength; ++i) {
51         for (ll j = 0; j < ylength; ++j) {
52             result[i + j] += xdigit[i]*
53                 ydigit[j];
54             if (result[i + j] >= 10) {
55                 result[i + j + 1] += result[
56                     i + j] / 10;
57                 result[i + j] %= 10;
58             }
59         }
60     }
61     ll start;
62     for (ll i = 2*n - 1; i >= 0; --i) {
63         if (result[i] != 0) {
64             start = i;
65             break;
66         }
67     }
68     string r = "";
69     for (ll i = start; i >= 0; --i) {
70         r += result[i] + '0';
71     }
72     return r;
73 }

```

7.2 Modular Inverse

```

1 // extend Euclidean
2 pll extgcd(ll a, ll b) {
3     if (b == 0) return pll(1, 0);
4     pll p = extgcd(m, a%b);
5     ll x = p.ff, y = p.ss;
6     return pll(y, x - y*(a/b));

```

```

7 }
8 extgcd(a, MOD).ff // the modular inverse
9 (extgcd(a, MOD).ff%MOD + MOD)%MOD // if you
  want to ensure that it is non-negative
10
11 // fast exponentiation (make sure that MOD
  is a prime number)
12 fastPow(a, MOD - 2) // the modular inverse

```

7.3 Fast Exponentiation

```

1 ll fastPow(ll b, ll e) {
2     b = b%MOD;
3     ll res = 1;
4     while (e > 0) {
5         if (e&2 == 1) res = (res*b)%MOD;
6         b = (b*b)%MOD;
7         e /= 2;
8     }
9     return res;
10 }

```

7.4 Matrix

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Matrix {
5     int n, m; // n 行, m 列
6     vector<vector<long long>>> a;
7     static const long long MOD = 1e9+7; //
  如果題目需要取模，可以改這裡
8
9     Matrix(int n, int m, bool ident = false)
10     : n(n), m(m) {
11         a.assign(n, vector<long long>(m, 0));
12         if(ident) { // 單位矩陣
13             for(int i=0; i<min(n,m); i++) a[i][i] = 1;
14         }
15     }
16
17 // 輸出矩陣
18 void print() const {
19     for(int i=0; i<n; i++) {
20         for(int j=0; j<m; j++) cout << a[i][j] << " ";
21         cout << "\n";
22     }
23 }
24
25 // 矩陣乘法
26 Matrix operator*(const Matrix& o) const
27 {
28     assert(m == o.n);
29     Matrix res(n, o.m);
30     for(int i=0; i<n; i++) {

```

```

29         for(int k=0; k<m; k++) if(a[i][k]
30             ]) {
31             for(int j=0; j<o.m; j++) {
32                 res.a[i][j] = (res.a[i][j]
33                     + a[i][k] * o.a[k]
34                     ][j]) % MOD;
35             }
36         }
37     }
38     return res;
39 }
40
41 // 矩陣加法
42 Matrix operator+(const Matrix& o) const
43 {
44     assert(n == o.n && m == o.m);
45     Matrix res(n, m);
46     for(int i=0; i<n; i++) {
47         for(int j=0; j<m; j++) {
48             res.a[i][j] = (a[i][j] + o.a[i][j]) % MOD;
49         }
50     }
51     return res;
52 }
53
54 // 矩陣快速幂 (n*n 方陣才能做)
55 Matrix pow(long long exp) const {
56     assert(n == m);
57     Matrix res(n, n, true), base = *this;
58     while(exp > 0) {
59         if(exp & 1) res = res * base;
60         base = base * base;
61         exp >>= 1;
62     }
63     return res;
64 }
65
66 };

```

7.5 Prime Sieve

```

1 // Sieve of Eratosthenes
2 ll MAXN;
3 vector<ll> spf(MAXN + 1, -1);
4 spf[0] = -2; spf[1] = -2;
5 for (ll i = 2; i*i <= MAXN; ++i) {
6     if (spf[i] == -1) {
7         for (ll j = i*i; j <= MAXN; j += i) {
8             if (spf[j] == -1) {
9                 spf[j] = i;
10            }
11        }
12    }
13 }

```

7.6 Combinations

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int M = 1e9 + 7;
5 const int N = 2000005;
6 vector<int> fact(N), invfact(N);
7
8 int qpow(int a, int b){
9     int res=1;
10    while(b > 0){
11        if(b & 1) res = res * a % M;
12        a = a * a % M;
13        b >>= 1;
14    }
15    return res;
16 }
17
18 void dofact(){
19     fact[0] = 1;
20     for (int i = 1; i < N; i++) {
21         fact[i] = fact[i - 1] * i % M;
22     }
23
24     invfact[N - 1] = qpow(fact[N - 1], M - 2);
25     for(int i = N - 1; i > 0; i--) invfact[i]
26         - 1] = invfact[i] * i % M;
27 }
28
29 int C(int n, int m){
30     return fact[n] * invfact[m] % M *
31         invfact[n - m] % M;
32 }
33
34 void solve() {
35 }
36
37 signed main() {
38     // FIO;
39     int t_t = 1;
40     // cin >> t_t;
41     while (t_t--) {
42         solve();
43     }
44 }
45
46 //
47 //
48
49 ll MAXN;
50
51 vector<ll> fact; // factorial
52 vector<ll> invs; // modular inverse
53 void init() {
54     fact.resize(MAXN + 1, 1);
55     invs.resize(MAXN + 1, 1);
56     for (ll i = 2; i <= MAXN; ++i) {
57         fact[i] = (fact[i - 1]*i)%MOD;
58         invs[i] = fastPow(fact[i], MOD - 2);
59     }

```

```

60 }
61
62 inline ll C(ll n, ll k) {
63     return fact[n]*invs[k]%MOD*invs[n - k]%MOD
64 }

```

8 Others

8.1 GCC Builtin Functions

```

1 // count the number of 1 bit in x
2 __builtin_popcount(unsigned int x)
3 __builtin_popcountll(unsigned long long x)
4
5 // count leading zero of x
6 __builtin_clz(unsigned int x)
7 __builtin_clzll(unsigned long long x)
8
9 // count trailing zero of x
10 __builtin_ctz(unsigned int x)
11 __builtin_ctzll(unsigned long long x)

```

8.2 mt19937 Usage

```

1 #include <chrono>
2 #include <random>
3 using namespace std;
4 using ll = long long;
5
6 // mt19937 is a random number generator
7 // chrono::steady_clock::now().
  time_since_epoch().count() is used as a
  seed
8 mt19937 rng(chrono::steady_clock::now().
  time_since_epoch().count());
9
10 // dist describes a range of random numbers
11 uniform_int_distribution<ll> dist(lb, ub);
12
13 dist(rng) // use rng to generate a random
  integer in [lb, ub]

```

8.3 distinct k number

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define all(c) (c).begin(), (c).end()
5
6 void solve() {
7     int n, k, L, R;
8     cin >> n >> k >> L >> R;
9     vector<int> uni(n);
10    vector<int> vec(n);
11    for (int i = 0; i < n; i++) {

```

```

12     cin >> uni[i];
13     vec[i] = uni[i];
14 }
15 sort(all(uni));
16 uni.resize(unique(all(uni)) - uni.begin
17 ());
18 vector<int> f_low(uni.size(), 0);
19 vector<int> f_up(uni.size(), 0);
20 for (int i = 0; i < n; i++) {
21     vec[i] = lower_bound(all(uni), vec[i]
22 ) - uni.begin();
23 }
24 int l = -1, r = 0;
25 int cl = 0, cr = 0;
26 int res = 0;
27 for (int i = 0; i < n; i++) {
28     while (r < n && cr <= k) {
29         if (f_up[vec[r]] == 0) {
30             if (cr == k) break;
31             cr++;
32         }
33         f_up[vec[r++]]++;
34     }
35     while (l + 1 < n && cl < k) {
36         if (f_low[vec[l + 1]] == 0) {
37             cl++;
38         }
39         f_low[vec[l + 1]]++;
40         l++;
41     }
42     if (cl == k) {
43         res += max(min(r, i + R) - max(l
44 , i + L - 1), 0LL);
45     }
46     f_low[vec[i]]--;
47     f_up[vec[i]]--;
48     if (f_low[vec[i]] == 0) cl--;
49     if (f_up[vec[i]] == 0) cr--;
50 }
51 cout << res << '\n';
52 }

```

```

16     static const uint64_t FIXED_RANDOM =
17         std::chrono::steady_clock::now
18         ().time_since_epoch().count();
19     return splitmix64(x + FIXED_RANDOM);
20 }
21 // for example:
22 // unordered_map<LL, LL, custom_hash> mp;
23 // =====
24 // =====
25 class custom_hash {
26 private:
27     static const uint64_t FIXED_RANDOM;
28     static uint64_t splitmix64(uint64_t x) {
29         x += 0x9e3779b97f4a7c15;
30         x = (x ^ (x >> 30)) * 0
31             xbf58476d1ce4e5b9;
32         x = (x ^ (x >> 27)) * 0
33             x94d049bb133111eb;
34         return x ^ (x >> 31);
35     }
36 public:
37     size_t operator()(uint64_t x) const {
38         return splitmix64(x + FIXED_RANDOM);
39     }
40     size_t operator()(pair<uint64_t,
41         uint64_t> p) const {
42         uint64_t h1 = splitmix64(p.first +
43             FIXED_RANDOM);
44         uint64_t h2 = splitmix64(p.second +
45             FIXED_RANDOM + 1);
46         return h1 ^ (h2*47);
47     }
48 };
49 const uint64_t custom_hash::FIXED_RANDOM =
50     std::chrono::steady_clock::now().
51     time_since_epoch().count();
52 // for example:
53 // unordered_map<pair<LL, LL, custom_hash> mp;

```

```

15     n = _s.size();
16     s = _s;
17
18     h1.resize(n); h1[0] = s[0];
19     h2.resize(n); h2[0] = s[0];
20     for (ll i = 1; i < n; ++i) {
21         h1[i] = (h1[i - 1]*p%m1 + s[i])%
22             m1;
23         h2[i] = (h2[i - 1]*p%m2 + s[i])%
24             m2;
25     }
26
27     p1.resize(n); p1[0] = 1;
28     p2.resize(n); p2[0] = 1;
29     for (ll i = 1; i < n; ++i) {
30         p1[i] = p1[i - 1]*p%m1;
31         p2[i] = p2[i - 1]*p%m2;
32     }
33     pll hash(ll l, ll r) const { // [l, r]
34         if (l == 0) return pll(h1[r], h2[r])
35             ;
36         return pll(
37             ((h1[r] - h1[l - 1]*p1[r - l +
38                 1]%m1)%m1 + m1)%m1,
39             ((h2[r] - h2[l - 1]*p2[r - l +
40                 1]%m2)%m2 + m2)%m2
41         );
42     }
43 };
44
45 int main() {
46     ios::sync_with_stdio(false);
47     cin.tie(nullptr);
48
49     string s1, s2; cin >> s1 >> s2;
50     ll n = s1.size(), m = s2.size();
51
52     if (n < m) {
53         cout << 0 << '\n';
54         return 0;
55     }
56
57     ll cnt = 0;
58     strHash sh1(s1), sh2(s2);
59     pll tarHash = sh2.hash(0, m - 1);
60     for (ll i = 0; i < n - m + 1; ++i) {
61         if (sh1.hash(i, i + m - 1) ==
62             tarHash) cnt += 1;
63     }
64     cout << cnt << '\n';
65
66     return 0;
67 }

```

```

5     vector<ll> suffix_array(const string& s) {
6         ll n = s.size();
7         vector<ll> sa(n), rk(n), tmp(n);
8
9         for (ll i = 0; i < n; ++i) {
10             sa[i] = i;
11             rk[i] = s[i] - 'a';
12         }
13
14         for (ll k = 1; k < n; k *= 2) {
15             auto cmp = [&](ll ri, ll li) {
16                 if (rk[ri] != rk[li]) return rk[
17                     ri] < rk[li];
18                 ll x = (ri + k < n) ? rk[ri + k]
19                     : -1;
20                 ll y = (li + k < n) ? rk[li + k]
21                     : -1;
22                 return x < y;
23             };
24             sort(sa.begin(), sa.end(), cmp);
25
26             tmp[sa[0]] = 0;
27             for (ll i = 1; i < n; ++i) {
28                 tmp[sa[i]] = tmp[sa[i - 1]] +
29                     cmp(sa[i - 1], sa[i]);
30             }
31             rk = tmp;
32         }
33         return sa;
34     }
35
36 int main() {
37     ios::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     string s; cin >> s;
41     vector<ll> sa = suffix_array(s);
42     ll n = s.size();
43
44     ll k; cin >> k;
45     for (ll i = 0; i < k; ++i) {
46         string tar; cin >> tar;
47         ll m = tar.size();
48
49         ll l = 0, r = n - 1;
50         while (l <= r) {
51             ll mid = l + (r - l)/2;
52             if (s.substr(sa[mid], m) < tar)
53                 {
54                     l = mid + 1;
55                 }
56             else {
57                 r = mid - 1;
58             }
59         }
60         cout << (s.substr(sa[l], m) == tar ?
61             "YES" : "NO") << '\n';
62     }
63     return 0;
64 }

```

8.4 Custom Hash

9 String

9.1 String Hashing

```

1 #include <chrono>
2
3 // =====
4 // =====
5
6 class custom_hash {
7 private:
8     static uint64_t splitmix64(uint64_t x) {
9         x += 0x9e3779b97f4a7c15;
10        x = (x ^ (x >> 30)) * 0
11            xbf58476d1ce4e5b9;
12        x = (x ^ (x >> 27)) * 0
13            x94d049bb133111eb;
14        return x ^ (x >> 31);
15    }
16 public:
17     size_t operator()(uint64_t x) const {

```

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 class strHash {
8 private:
9     const ll m1 = 1e9 + 7, m2 = 1e9 + 9, p =
10         239017;
11     ll n;
12     string s;
13     vector<ll> h1, h2, p1, p2;
14 public:
15     strHash(const string& _s) {

```

9.2 Suffix Array n log squared n version

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4

```

9.3 Trie

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Trie {
5     struct Node {
6         bool endofWord;
7         vector<int> children;
8         Node() : endofWord(false), children
9             (26, -1) {}
10    };
11    vector<Node> trie;
12
13 public:
14     Trie() {
15         trie.emplace_back();
16     }
17
18     void insert(const string& word) {
19         int cur = 0;
20         for (char c : word) {
21             int idx = c - 'a';
22             if (trie[cur].children[idx] ==
23                 -1) {
24                 trie[cur].children[idx] =
25                     trie.size();
26                 trie.emplace_back();
27             }
28             cur = trie[cur].children[idx];
29         }
30         trie[cur].endofWord = true;
31     }
32
33     bool search(const string& word) {
34         int cur = 0;
35         for (char c : word) {
36             int idx = c - 'a';
37             if (trie[cur].children[idx] ==
38                 -1) return false;
39             cur = trie[cur].children[idx];
40         }
41         return trie[cur].endofWord;
42     }
43
44     bool startsWith(const string& prefix) {
45         int cur = 0;
46         for (char c : prefix) {
47             int idx = c - 'a';
48             if (trie[cur].children[idx] ==
49                 -1) return false;
50             cur = trie[cur].children[idx];
51         }
52         return true;
53     }
54
55     void deleteWord(const string& word) {
56         int cur = 0;
57         for (char c : word) {
58             int idx = c - 'a';
59             if (trie[cur].children[idx] ==
60                 -1) return;
61             cur = trie[cur].children[idx];
62         }
63     }
64 }

```

```

58     trie[cur].endofWord = false;
59 }
60
61 void print(int node, string prefix)
62     const {
63     if (trie[node].endofWord) {
64         cout << prefix << "\n";
65     }
66     for (int i = 0; i < 26; i++) {
67         int nxt = trie[node].children[i];
68         if (nxt != -1) {
69             print(nxt, prefix + char('a'
70                 + i));
71         }
72     }
73 }
74
75 void print() const { print(0, ""); }
76
77 int main() {
78     Trie trie;
79
80     trie.insert("geek");
81     trie.insert("geeks");
82     trie.insert("code");
83     trie.insert("coder");
84     trie.insert("coding");
85
86     cout << "Trie contents:\n";
87     trie.print();
88
89     cout << "\nSearch results:\n";
90     cout << "geek: " << trie.search("geek")
91         << "\n";
92     cout << "geeks: " << trie.search("geeks")
93         << "\n";
94     cout << "code: " << trie.search("code")
95         << "\n";
96     cout << "coder: " << trie.search("coder")
97         << "\n";
98     cout << "coding: " << trie.search("coding")
99         << "\n";
100     cout << "codex: " << trie.search("codex")
101         << "\n";
102
103     cout << "\nPrefix results:\n";
104     cout << "ge: " << trie.startsWith("ge")
105         << "\n";
106     cout << "cod: " << trie.startsWith("cod")
107         << "\n";
108     cout << "coz: " << trie.startsWith("coz")
109         << "\n";
110
111     trie.deleteWord("coding");
112     trie.deleteWord("geek");
113
114     cout << "\nTrie contents after deletions
115         :\n";
116     trie.print();
117
118     cout << "\nSearch results after
119         deletions:\n";
120     cout << "coding: " << trie.search("coding")
121         << "\n";

```

```

109     cout << "geek: " << trie.search("geek")
110         << "\n";
111
112     return 0;

```

9.4 KMP

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 template<class T> using vec = vector<T>;
6 #define endl '\n'
7 #define ff first
8 #define ss second
9 #define pb push_back
10 #define rep(n) for (ll _ = 1; _ <= n; ++_)
11
12 // -----
13
14 vector<ll> LPS(const string& s) { // 0-
15     indexed
16     ll n = s.size();
17     if (n == 0) return {};
18     vector<ll> lps(n); lps[0] = 0;
19     for (ll i = 1, j = 0; i < n; ++i) {
20         while (j > 0 && s[i] != s[j]) j =
21             lps[j - 1];
22         if (s[i] == s[j]) j += 1;
23         lps[i] = j;
24     }
25     return lps;
26 }
27
28 ll KMP(const string& s, const string& t) {
29     // 0-indexed
30     vector<ll> lps = LPS(t);
31     ll n = s.size(), m = t.size();
32     if (n < m) return 0;
33     ll i = 0, j = 0, cnt = 0;
34     while (i < n) {
35         if (s[i] == t[j]) {
36             i += 1;
37             j += 1;
38         } else if (j > 0) {
39             j = lps[j - 1];
40         } else if (j == 0) {
41             i += 1;
42         }
43         if (j == m) {
44             // s[i - m to i - 1] == t
45             cnt += 1;
46             j = lps[j - 1];
47         }
48     }
49     return cnt; // the number of matching
50     substrings
51 }
52
53 int main() {

```

```

50     ios::sync_with_stdio(false);
51     cin.tie(nullptr);
52
53     string s, t;
54     cin >> s >> t;
55
56     cout << KMP(s, t) << endl;
57
58     return 0;
59 }

```

9.5 Z-value

```

1 // CSES: String Matching
2 // Given a string and a pattern, your task
3 // is to count
4 // the number of positions where the pattern
5 // occurs in the string.
6
7 #include <bits/stdc++.h>
8 using namespace std;
9 using ll = long long;
10
11 vector<ll> z_value(const string& s) {
12     ll n = s.size();
13     vector<ll> z(n);
14     ll l = 0, r = 0;
15     for (ll i = 1; i < n; ++i) {
16         if (i <= r) z[i] = min(z[i - l], r -
17             i + 1);
18         while (i + z[i] < n && s[z[i]] == s[
19             i + z[i]]) z[i] += 1;
20         if (i + z[i] - 1 > r) {
21             l = i;
22             r = i + z[i] - 1;
23         }
24     }
25     return z;
26 }
27
28 int main() {
29     ios::sync_with_stdio(false);
30     cin.tie(nullptr);
31
32     string s1, s2; cin >> s1 >> s2;
33     ll n = s1.size(), m = s2.size();
34
35     ll cnt = 0;
36     string s = s2 + "$" + s1;
37     vector<ll> z = z_value(s);
38     for (ll i = m; i < s.size(); ++i) {
39         if (z[i] == m) cnt += 1;
40     }
41     cout << cnt << '\n';
42
43     return 0;
44 }
45
46 // =====
47 // =====
48
49 // CSES: Finding Borders

```

```

46 // A border of a string is a prefix that is
    // also a suffix of
47 // the string but not the whole string. For
    // example,
48 // the borders of abcababcbab are ab and
    // abcab.
49 // Your task is to find all border lengths
    // of a given string.

50
51 #include <bits/stdc++.h>
52 using namespace std;
53 using ll = long long;
54 #define pb push_back
55
56 vector<ll> z_value(const string& s) {
57     ll n = s.size();
58     vector<ll> z(n);
59     ll l = 0, r = 0;
60     for (ll i = 1; i < n; ++i) {
61         if (i <= r) z[i] = min(z[i - l], r -
62                               i + 1);
63         while (i + z[i] < n && s[z[i]] == s[
64               i + z[i]]) z[i] += 1;
65         if (i + z[i] - 1 > r) {
66             l = i;
67             r = i + z[i] - 1;
68         }
69     }
70     return z;
71 }
72
73 int main() {
74     ios::sync_with_stdio(false);
75     cin.tie(nullptr);
76
77     string s; cin >> s;
78     ll n = s.size();
79
80     vector<ll> z = z_value(s), res;
81     for (ll i = 1; i < n; ++i) {
82         if (i + z[i] == n) res.pb(z[i]);
83     }
84     sort(res.begin(), res.end());
85     for (ll x : res) {
86         cout << x << ' ';
87     } cout << '\n';
88     return 0;
89 }
90 // =====
91 // =====
92
93 // CSES: Finding Periods
94 // A period of a string is a prefix that can
    // be used to generate
95 // the whole string by repeating the prefix.
    // The last repetition
96 // may be partial. For example, the periods
    // of abcabca are abc, abcab and abcabca.
97 // Your task is to find all period lengths
    // of a string.
98
99 #include <bits/stdc++.h>
100 using namespace std;
101 using ll = long long;

```

```

102 #define pb push_back
103
104 vector<ll> z_value(const string& s) {
105     ll n = s.size();
106     vector<ll> z(n);
107     ll l = 0, r = 0;
108     for (ll i = 1; i < n; ++i) {
109         if (i <= r) z[i] = min(z[i - l], r -
110                               i + 1);
111         while (i + z[i] < n && s[z[i]] == s[
112               i + z[i]]) z[i] += 1;
113         if (i + z[i] - 1 > r) {
114             l = i;
115             r = i + z[i] - 1;
116         }
117     }
118     return z;
119 }
120
121 int main() {
122     ios::sync_with_stdio(false);
123     cin.tie(nullptr);
124
125     string s; cin >> s;
126     ll n = s.size();
127
128     vector<ll> z = z_value(s), res;
129     for (ll i = 1; i < n; ++i) {
130         if (i + z[i] == n) res.pb(i);
131     }
132     sort(res.begin(), res.end());
133     for (ll x : res) {
134         cout << x << ' ';
135     } cout << '\n';
136     return 0;
137 }

```

9.6 Manacher

```

1 // CSES: Longest Palindrome
2 // Given a string, your task is to determine
    // the longest
3 // palindromic substring of the string. For
    // example,
4 // the longest palindrome in aybabtu is bab.
5
6 #include <bits/stdc++.h>
7 using namespace std;
8 using ll = long long;
9
10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     string t, s = "^#"; cin >> t;
15     ll n = t.size(), m = 2*n + 3;
16     for (ll i = 0; i < n; ++i) {
17         s += t[i];
18         s += (i == n - 1 ? "$" : "#");
19     }
20
21     ll c = 1;

```

```

22 vector<ll> p(m); p[1] = 0;
23 for (ll i = 2; i <= m - 3; ++i) {
24     if (i < c + p[c]) p[i] = min(p[c - (
25         i - c)], (c + p[c]) - i);
26     while (i + p[i] + 1 < m &&
27           i - p[i] - 1 >= 0 &&
28           s[i - p[i] - 1] == s[i + p[i]
29             + 1]) p[i] += 1;
30     if (i + p[i] > c + p[c]) c = i;
31 }
32
33 ll j = 2;
34 for (ll i = 3; i <= m - 3; ++i) {
35     if (p[i] > p[j]) j = i;
36 }
37
38 for (ll i = j - p[j] + 1; i <= j + p[j]
39       - 1; i += 2) {
40     cout << s[i];
41 } cout << '\n';
42
43 return 0;
44 // =====
45 // =====
46
47 // CSES: All Palindromes
48 // Given a string, calculate for each
    // position the length
49 // of the longest palindrome that ends at
    // that position.
50
51 #include <bits/stdc++.h>
52 using namespace std;
53 using ll = long long;
54
55 ll n, m;
56 string ori_s, s;
57 vector<ll> p, rt, dp;
58
59 int main() {
60     ios::sync_with_stdio(false);
61     cin.tie(nullptr);
62
63     cin >> ori_s;
64     n = ori_s.size();
65     m = 2*n + 3;
66     s = "^#";
67     for (ll i = 0; i < n; ++i) {
68         s += ori_s[i];
69         s += (i == n - 1 ? "$" : "#");
70     }
71
72     ll c = 0;
73     p.resize(m);
74     for (ll i = 2; i <= m - 3; ++i) {
75         if (i < c + p[c]) p[i] = min(p[c - (
76             i - c)], (c + p[c]) - i);
77         while (i + p[i] + 1 < m &&
78               i - p[i] - 1 >= 0 &&
79               s[i - p[i] - 1] == s[i + p[i]
80                 + 1]) p[i] += 1;
81         if (i + p[i] > c + p[c]) c = i;
82     }
83 }

```

```

81 rt.resize(n, 1);
82 for (ll i = 2; i <= m - 3; ++i) {
83     ll y = ((i + p[i] - 1) - 2)/2;
84     if (s[i] == '#') {
85         ll x = ((i + 1) - 2)/2;
86         rt[y] = max(rt[y], (y - x + 1)
87                     * 2);
88     } else {
89         ll x = (i - 2)/2;
90         rt[y] = max(rt[y], (y - x)*2 +
91                     1);
92     }
93 }
94
95 dp.resize(n); dp[n - 1] = rt[n - 1];
96 for (ll i = n - 2; i >= 0; --i) {
97     dp[i] = max(rt[i], dp[i + 1] - 2);
98 }
99
100 for (ll x : dp) {
101     cout << x << ' ';
102 } cout << '\n';
103
104 return 0;
105 }

```

10 Tree

10.1 Binary Lifting

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 inline ll flg(ll x) {
6     return 63 - __builtin_clzll(x);
7 }
8
9 inline bool isOnBit(ll x, ll i) {
10     return ((1LL << i) & x) > 0;
11 }
12
13 ll n, q, lgn;
14 vector<vector<ll>> blf;
15
16 void init() {
17     blf[0][1] = -1;
18     for (ll u = 2; u <= n; ++u) cin >> blf
19         [0][u];
20
21     for (ll h = 1; h <= lgn; ++h) {
22         for (ll u = 1; u <= n; ++u) {
23             ll nt = blf[h - 1][u];
24             blf[h][u] = nt == -1 ? -1 : blf[
25                 h - 1][nt];
26         }
27     }
28
29     ll query(ll u, ll step) {
30         ll cur = u;
31         for (ll i = 30; i >= 0; --i) {

```



```

30     if (isOnBit(step, i)) {
31         cur = blf[i][cur];
32         if (cur == -1) return -1;
33     }
34 }
35 return cur;
36 }
37
38 int main() {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     cin >> n >> q;
43     lgn = flg(n);
44     blf.resize(lgn + 1, vector<ll>(n + 1));
45     init();
46
47     while (q--) {
48         ll u, step; cin >> u >> step;
49         cout << query(u, step) << '\n';
50     }
51
52     return 0;
53 }

```

10.2 LCA

```

1 // Use binary Lifting
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 using ll = long long;
6 #define pb push_back
7
8 inline ll flg(ll x) {
9     return 63 - __builtin_clzll(x);
10 }
11
12 inline bool isOnBit(ll x, ll i) {
13     return ((1LL << i) & x) > 0;
14 }
15
16 ll n, q, lgn;
17 vector<ll> d;
18 vector<vector<ll>> blf, adj;
19
20 void init() {
21     blf[0][1] = -1;
22     for (ll u = 2; u <= n; ++u) {
23         ll v; cin >> v;
24         blf[0][u] = v;
25         adj[v].pb(u);
26     }
27
28     for (ll h = 1; h <= lgn; ++h) {
29         for (ll u = 1; u <= n; ++u) {
30             ll nt = blf[h - 1][u];
31             blf[h][u] = nt == -1 ? -1 : blf[
32                 h - 1][nt];
33         }
34     }
35
36     ll query(ll u, ll step) {

```

```

36     ll cur = u;
37     for (ll i = 30; i >= 0; --i) {
38         if (isOnBit(step, i)) {
39             cur = blf[i][cur];
40             if (cur == -1) return -1;
41         }
42     }
43     return cur;
44 }
45
46 void dfs(ll u, ll dn) {
47     d[u] = dn;
48     for (ll v : adj[u]) {
49         dfs(v, dn + 1);
50     }
51 }
52
53 ll lca(ll u, ll v) {
54     if (d[u] > d[v]) swap(u, v);
55     if (d[u] < d[v]) v = query(v, d[v] - d[u]);
56
57     if (u == v) return u;
58     for (ll h = lgn; h >= 0; --h) {
59         ll ntu = blf[h][u];
60         ll ntv = blf[h][v];
61         if (ntu == -1 || ntv == -1 || ntu ==
62             ntv) continue;
63         u = ntu;
64         v = ntv;
65     }
66     return blf[0][u];
67 }
68
69 int main() {
70     ios::sync_with_stdio(false);
71     cin.tie(nullptr);
72
73     cin >> n >> q;
74     lgn = flg(n);
75     d.resize(n + 1);
76     blf.resize(lgn + 1, vector<ll>(n + 1));
77     adj.resize(n + 1);
78     init();
79     dfs(1, 0);
80
81     while (q--) {
82         ll u, v; cin >> u >> v;
83         cout << lca(u, v) << '\n';
84     }
85
86     return 0;
87 }

```

ACM ICPC Team Reference - c0mpile_error

Contents

1 Bitwise Trick	1	3 D&C	2	6.5 Dijkstra	7	7.5 Prime Sieve	11
1.1 tricks	1	3.1 MergeSort Finds the Number of Inversions	2	6.6 Tarjan 縮點	8	7.6 Combinations	11
2 DP	1	4 Data Structure	2	6.7 Dinic 無權二分圖最大匹配	8	8 Others	11
2.1 Digit DP	1	4.1 DSU	2	6.8 Floyd Warshall	8	8.1 GCC Builtin Functions	11
2.2 Interval DP	1	4.2 Segment Tree	2	6.9 MST Prim	8	8.2 mt19937 Usage	11
2.3 Knapsack	2	4.3 BIT	3	6.10 MST Kruskal	9	8.3 distinct k number	11
2.4 Subset DP	2	4.4 Treap	3	6.11 Topo-sort DFS	9	8.4 Custom Hash	12
2.5 LIS	2	4.5 Sparse Table	5	6.12 Topo-sort Kahn	9	9 String	12
2.6 LCS	2	5 Geometry	6	6.13 Tarjan 2-SAT	9	9.1 String Hashing	12
		5.1 Convex Hull	6	6.14 Bridge	9	9.2 Suffix Array n log squared n version	12
		5.2 Vector	6	6.15 Bellman Ford Detects Negative Cycle	10	9.3 Trie	13
		5.3 Simple Polygon Area	6	6.16 Contracting Vertices Based on Specific Conditions	10	9.4 KMP	13
		5.4 Line Segment Intersection Test	6	6.17 Tarjan	10	9.5 Z-value	13
		6 Graph	6	7 Math	10	9.6 Manacher	14
		6.1 Kosaraju	6	7.1 Big Integer Addition and Multiplication	10	10 Tree	14
		6.2 Dinic Min Cut	6	7.2 Modular Inverse	10	10.1 Binary Lifting	14
		6.3 AP	7	7.3 Fast Exponentiation	11	10.2 LCA	15
		6.4 Dinic	7	7.4 Matrix	11		

ACM ICPC Judge Test - c0mpile_error

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6     const size_t KB = 1024;
7     const size_t MB = KB * 1024;
8     const size_t GB = MB * 1024;
9
10    size_t block_size, bound;
11    void stack_size_dfs(size_t depth = 1) {

```

```

12        if (depth >= bound)
13            return;
14        int8_t ptr[block_size]; // 若無法編譯將
15            block_size 改成常數
16        memset(ptr, 'a', block_size);
17        cout << depth << endl;
18        stack_size_dfs(depth + 1);
19    }
20    void stack_size_and_runtime_error(size_t
21        block_size, size_t bound = 1024) {
22        system_test::block_size = block_size;
23        system_test::bound = bound;
24        stack_size_dfs();
25    }
26    double speed(int iter_num) {
27        const int block_size = 1024;
28        volatile int A[block_size];
29        auto begin = chrono::high_resolution_clock
30            ::now();
31        while (iter_num--)
32            for (int j = 0; j < block_size; ++j)
33                A[j] += j;
34        auto end = chrono::high_resolution_clock::
35            now();
36        chrono::duration<double> diff = end -
37            begin;

```

```

38        return diff.count();
39    }
40    void runtime_error_1() {
41        // Segmentation fault
42        int *ptr = nullptr;
43        *(ptr + 7122) = 7122;
44    }
45    void runtime_error_2() {
46        // Segmentation fault
47        int *ptr = (int *)memset;
48        *ptr = 7122;
49    }
50    void runtime_error_3() {
51        // munmap_chunk(): invalid pointer
52        int *ptr = (int *)memset;
53        delete ptr;
54    }
55    void runtime_error_4() {
56        // free(): invalid pointer
57        int *ptr = new int[7122];
58        ptr += 1;
59        delete[] ptr;
60    }
61    }
62

```

```

63    void runtime_error_5() {
64        // maybe illegal instruction
65        int a = 7122, b = 0;
66        cout << (a / b) << endl;
67    }
68    void runtime_error_6() {
69        // floating point exception
70        volatile int a = 7122, b = 0;
71        cout << (a / b) << endl;
72    }
73    void runtime_error_7() {
74        // call to abort.
75        assert(false);
76    }
77    } // namespace system_test
78
79    #include <sys/resource.h>
80    void print_stack_limit() { // only work in
81        Linux
82        struct rlimit l;
83        getrlimit(RLIMIT_STACK, &l);
84        cout << "stack_size = " << l.rlim_cur << "
85            byte" << endl;
86    }
87

```